

Marca de agua dinámica

Tamaño de píxeles en tiempo real

Cómo funciona el control del **tamaño (px)**, color y grosor de la marca de agua sobre el reproductor, editable en vivo desde el panel admin y aplicado a todos los reproductores conectados sin recargar.

[Frontend / Reproductor](#)

[Backend / SSE](#)

[Panel Admin](#)

[Código completo](#)

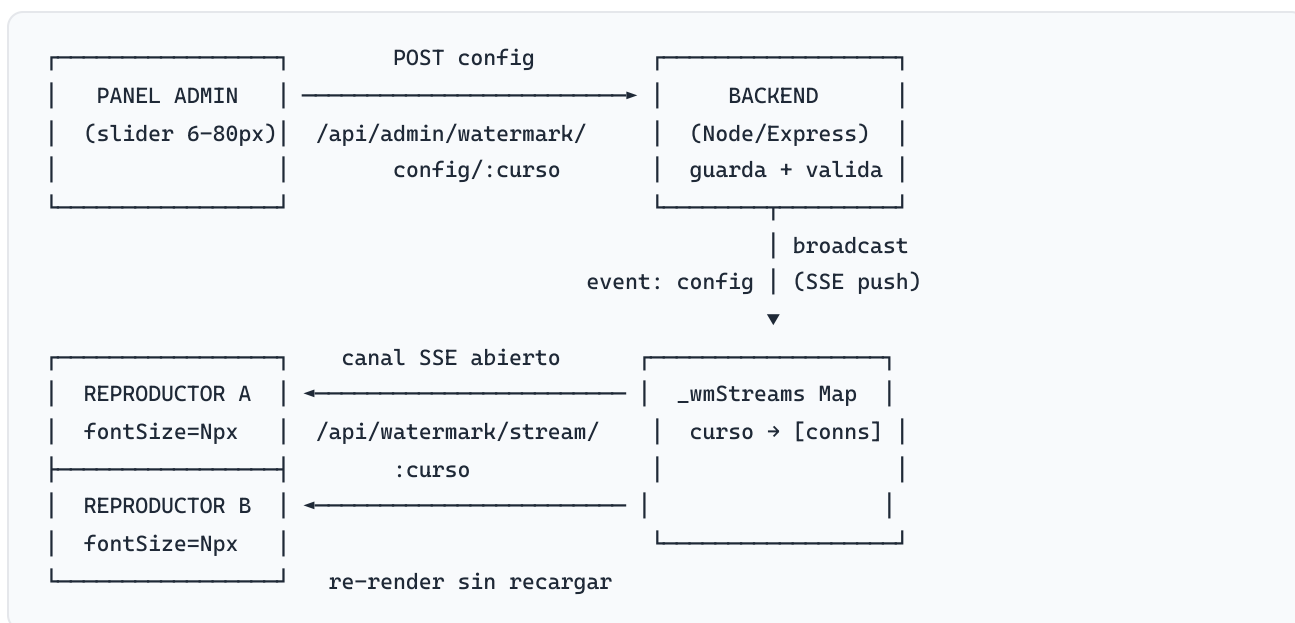
Guía de implementación para portarlo a otro reproductor y otra página · Edulock Systems

1. Qué es y cómo funciona

El sistema pinta una **marca de agua** (correo, IP, código, fecha/hora) encima del video. Cada marca tiene tres propiedades visuales configurables: **tamaño en píxeles**, **color** y **grosor**. El punto clave es que estas propiedades — muy especialmente el **tamaño en píxeles** — se pueden cambiar **en tiempo real** desde el panel de administración y el cambio llega a todos los reproductores que están reproduciendo en ese momento, **sin recargar la página**.

La comunicación en tiempo real se hace con **SSE (Server-Sent Events)**: el reproductor abre un canal persistente al servidor y este le empuja la configuración nueva cada vez que el admin la modifica.

Arquitectura de 3 capas



Flujo paso a paso

- Al abrir el video, el backend envía al reproductor la config inicial (`watermarkConfig`) dentro de la respuesta de reproducción.
- El reproductor pinta cada marca con `el.style.fontSize = size + 'px'`.
- El reproductor abre un canal **SSE** a `/api/watermark/stream/:curso`.
- El admin mueve el slider de **Tamaño** (6-80 px) → auto-guardado con debounce (700 ms).
- El backend guarda, valida (clamp 6-80) y hace **broadcast** por SSE a los reproductores del mismo curso.
- El reproductor recibe `event: config`, actualiza su config y **re-renderiza** las marcas con el nuevo tamaño en píxeles — sin recargar.

Idea central: el "tamaño de píxeles en tiempo real" es simplemente la propiedad CSS `font-size` (en px) de cada marca, cuyo valor viaja del admin al reproductor por un canal SSE persistente y se reaplica al vuelo.

Modelo de datos de la configuración

La config está organizada por **Sistema Operativo** (para que cada plataforma tenga su propio tamaño) y dentro por **marca**:

```
// Config resuelta que recibe el reproductor (ejemplo para 'web')
{
  "web": {
    "enabled": true,
    "email": { "on":true, "size":16, "color":"#ffffff", "weight":600 },
    "ip": { "on":true, "size":16, "color":"#ffffff", "weight":600 },
    "code": { "on":true, "size":16, "color":"#ffffff", "weight":600 },
    "datetime": { "on":true, "size":16, "color":"#ffffff", "weight":600 }
  },
  "windows": { /* email.size = 35, etc. */ },
  "android": { /* email.size = 22, etc. */ }
  // ... mac, linux, ios
}
```

Campo	Tipo	Rango / valores	Significado
enabled	bool	true / false	Activa las marcas para ese SO
on	bool	true / false	Marca visible o no
size	int	6 – 80	Tamaño en píxeles (font-size)
color	string	#rrggbb	Color del texto
weight	int	100 – 900	Grosor de la fuente

2. Backend — config + validación + SSE

Node.js + Express. Guarda la config, valida el tamaño en píxeles y la empuja en tiempo real por SSE.

2.1 · Definición y validación (el clamp del tamaño)

El tamaño en píxeles se limita al rango **6–80** para que un valor inválido nunca rompa el reproductor. Esta es la pieza de seguridad clave del "tamaño en tiempo real".

```

const WM_OS_LIST = ['web', 'windows', 'mac', 'linux', 'android', 'ios'];
const WM_MARKS    = ['email', 'ip', 'code', 'datetime'];

// Valores por defecto por SO (para que nada cambie hasta que el admin edite)
function _wmDefaultMark(os, key) {
  const isDesktop = (os === 'windows' || os === 'mac' || os === 'linux');
  const isMobile  = (os === 'android' || os === 'ios');
  if (key === 'email') {
    if (isDesktop) return { on:true, size:35, color:'#ff2d2d', weight:700 };
    if (isMobile)  return { on:true, size:22, color:'#ff2d2d', weight:700 };
    return { on:true, size:16, color:'#ffffff', weight:600 }; // web
  }
  if (isMobile) return { on:true, size:11, color:'#ffffff', weight:600 };
  return { on:true, size:16, color:'#ffffff', weight:600 };
}

function buildDefaultWmOs(os) {
  const o = { enabled: true };
  WM_MARKS.forEach(k => { o[k] = _wmDefaultMark(os, k); });
  return o;
}

function buildDefaultWmConfig() {
  const c = {};
  WM_OS_LIST.forEach(os => { c[os] = buildDefaultWmOs(os); });
  return c;
}

// — Saneos: aquí se fuerza el rango del TAMAÑO EN PÍXELES (6-80) —
function _wmClampInt(v, min, max, def) {
  const n = parseInt(v, 10);
  if (!Number.isFinite(n)) return def;
  return Math.min(max, Math.max(min, n));
}

function _wmSanColor(v, def) {
  return (typeof v === 'string' && /^[0-9a-fA-F]{6}$/.test(v)) ? v.toLowerCase() : def;
}

function _wmSanMark(input, os, key) {
  const d = _wmDefaultMark(os, key);
  input = input || {};
  return {
    on:    input.on === undefined ? d.on : !!input.on,
    size:  _wmClampInt(input.size, 6, 80, d.size), // ◀ TAMAÑO px 6-80
    color: _wmSanColor(input.color, d.color),
    weight: _wmClampInt(input.weight, 100, 900, d.weight),
  };
}

function _wmSanOs(input, os) {
  const d = buildDefaultWmOs(os);
  input = input || {};
  const o = { enabled: input.enabled === undefined ? d.enabled : !!input.enabled };
  WM_MARKS.forEach(k => { o[k] = _wmSanMark(input[k], os, k); });
  return o;
}

function sanitizeWmConfig(input) {
  const out = {};
  WM_OS_LIST.forEach(os => { out[os] = _wmSanOs(input && input[os], os); });
  return out;
}

```

2.2 · Persistencia y resolución con herencia por curso

```
async function loadWmConfigs() {
  try {
    const raw = await db.getConfig('watermark_configs');
    if (!raw) return {};
    const obj = JSON.parse(raw);
    return (obj && typeof obj === 'object') ? obj : {};
  } catch { return {}; }
}

async function saveWmConfigs(obj) {
  await db.setConfig('watermark_configs', JSON.stringify(obj || {}));
}

// Config de un curso, mezclada con defaults. Si el curso no tiene, hereda __default__
function resolveWmConfig(configs, courseId) {
  const key = courseId || '__default__';
  const stored = (configs && configs[key]) || (configs && configs['__default__']) || null;
  if (!stored) return buildDefaultWmConfig();
  return sanitizeWmConfig(stored);
}
```

2.3 · Registro de suscriptores SSE y broadcast

```
// courseKey → Set<res> (conexiones SSE abiertas por curso)
const _wmStreams = new Map();

function _wmSubscribe(courseKey, res) {
  if (!_wmStreams.has(courseKey)) _wmStreams.set(courseKey, new Set());
  _wmStreams.get(courseKey).add(res);
}

function _wmUnsubscribe(courseKey, res) {
  const set = _wmStreams.get(courseKey);
  if (set) { set.delete(res); if (set.size === 0) _wmStreams.delete(courseKey); }
}

// Empuja la config nueva a los reproductores conectados de ese curso
async function broadcastWmConfig(courseKey) {
  const configs = await loadWmConfigs();
  const set = _wmStreams.get(courseKey);
  if (!set || set.size === 0) return;
  const cfg = resolveWmConfig(configs, courseKey);
  const data = JSON.stringify(cfg);
  for (const res of set) {
    try { res.write(`event: config\ndata: ${data}\n\n`); } catch {}
  }
}
```

Herencia por curso (opcional): cuando se edita el curso `__default__`, el broadcast recorre **todos** los canales conectados y empuja el default a los cursos que no tienen config propia. Si no necesitas cursos, puedes usar un único canal global.

2.4 · Rutas Express (SSE + lectura + guardado admin)

```
// — Canal SSE: empuja la config en tiempo real —
// EventSource no envía cabeceras → auth por query token
app.get('/api/watermark/stream/:courseId', async (req, res) => {
  let payload = null;
  try { payload = jwt.verify(String(req.query.token || ''), JWT_SECRET); } catch { payload = null; }
  if (!payload) return res.status(401).end();

  const courseKey = req.params.courseId || '__default__';
  res.set({
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache, no-transform',
    'Connection': 'keep-alive',
    'X-Accel-Buffering': 'no', // desactiva buffering de nginx
  });
  if (typeof res.flushHeaders === 'function') res.flushHeaders();

  // 1) Enviar config inicial inmediatamente
  const configs = await loadWmConfigs();
  const cfg = resolveWmConfig(configs, courseKey === '__default__' ? null : courseKey);
  res.write(`event: config\\ndata: ${JSON.stringify(cfg)}\\n\\n`);

  // 2) Suscribirse + latido cada 25s para mantener viva la conexión
  _wmSubscribe(courseKey, res);
  const hb = setInterval(() => { try { res.write(': ping\\n\\n'); } catch {} }, 25000);
  req.on('close', () => { clearInterval(hb); _wmUnsubscribe(courseKey, res); });
});

// — Admin: leer todo para el editor —
app.get('/api/admin/watermark/configs', requireAdmin, async (req, res) => {
  const configs = await loadWmConfigs();
  let courses = [];
  try { courses = (await db.getAllCourses()).map(c => ({ id:c.id, name:c.name })); } catch {}
  res.json({ osList: WM_OS_LIST, marks: WM_MARKS,
    defaults: buildDefaultWmConfig(), courses, configs });
});

// — Admin: guardar + broadcast en tiempo real —
app.post('/api/admin/watermark/config/:courseId', requireAdmin, async (req, res) => {
  const courseKey = req.params.courseId || '__default__';
  const clean = sanitizeWmConfig((req.body && req.body.config) || {});
  const configs = await loadWmConfigs();
  configs[courseKey] = clean;
  await saveWmConfigs(configs);
  broadcastWmConfig(courseKey).catch(() => {}); // ◀ push a reproductores
  res.json({ ok: true, courseId: courseKey, config: clean });
});
```

Nginx / proxy: para que el SSE fluya sin buffering añade en el bloque `location` `proxy_buffering off;` y `proxy_read_timeout 3600s;`. La cabecera `X-Accel-Buffering: no` ya lo desactiva por respuesta.

3. Reproductor — render y SSE

Aquí ocurre el "tamaño de píxeles en tiempo real": cada marca se pinta con `fontSize = size + 'px'` y se re-renderiza al recibir config nueva.

3.1 · Estilo de cada marca (lee el tamaño de la config)

```
// Config de marca de agua (llega del backend, editable en tiempo real)
let _wmConfig = null; // { email:{on,size,color,weight}, ip:{...}, ... }
let _wmOsKey = 'web';

// Devuelve el estilo aplicable de una marca. Fallback = look por defecto.
function _wmMarkStyle(key) {
  const def = { on:true, size:16, color:'#ffffff', weight:600 };
  if (!_wmConfig || typeof _wmConfig !== 'object') return def;
  if (_wmConfig.enabled === false) return { ...def, on:false };
  const m = _wmConfig[key];
  if (!m) return def;
  return {
    on: m.on !== false,
    size: parseInt(m.size, 10) || def.size, // ← tamaño en px
    color: (typeof m.color === 'string' && /^[0-9a-fA-F]{6}$/.test(m.color)) ? m.color : def.color,
    weight: parseInt(m.weight, 10) || def.weight,
  };
}
```

3.2 · Pintar las marcas — aquí se aplica el tamaño en píxeles

```
// Definición de las 4 marcas: cada una con su zona, tiempo y dato
const WM_DEFS = [
  { id:'wm-email',    key:'email',    rotateMs:9000, transSec:6,
    zone:[ {top:'9%',left:'6%'}, {top:'19%',left:'13%'}, {top:'7%',left:'21%'} ] },
  { id:'wm-ip',      key:'ip',      rotateMs:13000, transSec:8,
    zone:[ {top:'10%',right:'6%'}, {top:'21%',right:'12%'} ] },
  { id:'wm-code',    key:'code',    rotateMs:7000, transSec:5,
    zone:[ {bottom:'14%',left:'6%'}, {bottom:'25%',left:'13%'} ] },
  { id:'wm-datetime', key:'datetime', rotateMs:11000, transSec:7,
    zone:[ {bottom:'14%',right:'6%'}, {bottom:'23%',right:'12%'} ] },
];

let _wmTimers = [], _wmEls = [];

function renderWatermark() {
  const parent = document.getElementById('player-container');
  if (!parent) return;

  // Limpiar timers/elementos previos (para re-render al llegar config nueva)
  _wmTimers.forEach(t => clearInterval(t)); _wmTimers = [];
  _wmEls.forEach(el => { try { el.remove(); } catch {} }); _wmEls = [];

  WM_DEFS.forEach(def => {
    const st = _wmMarkStyle(def.key);
    let el = document.getElementById(def.id);
    if (!st.on) { if (el) el.remove(); return; }
    if (!el) { el = document.createElement('div'); el.id = def.id; parent.appendChild(el); }

    // ★ APLICACIÓN DEL TAMAÑO EN PÍXELES EN TIEMPO REAL ★
    el.style.position = 'absolute';
    el.style.color = st.color;
    el.style.fontSize = st.size + 'px'; // ← tamaño de píxeles
    el.style.fontWeight = String(st.weight);
    el.style.fontFamily = 'monospace';
    el.style.textShadow = '1px 1px 2px rgba(0,0,0,0.8)';
    el.style.whiteSpace = 'nowrap';
    el.style.userSelect = 'none';
    el.style.pointerEvents = 'none';
    el.style.zIndex = '20';
    el.style.opacity = '0.22';
    el.style.transition = `top ${def.transSec}s ease, left ${def.transSec}s ease, right ${def.transSec}s ease`;

    // Texto y posición inicial (rotación anti-recorte)
    el.textContent = wmValue(def.key);
    let idx = Math.floor(Math.random() * def.zone.length);
    applyPos(el, def.zone[idx]);

    const timer = setInterval(() => {
      el.textContent = wmValue(def.key); // refresca (hora, etc.)
      idx = (idx + 1) % def.zone.length;
      applyPos(el, def.zone[idx]);
    }, def.rotateMs);
    _wmTimers.push(timer); _wmEls.push(el);
  });
}

function applyPos(el, p) {
  el.style.top = p.top||''; el.style.left = p.left||'';
```

```
    el.style.right = p.right||''; el.style.bottom = p.bottom||'';  
  }
```

3.3 · Valores de cada marca (correo, IP, código, fecha)

```
let _studentEmail = null, _studentCode = null, _clientIp = null;

function wmValue(key) {
  if (key === 'email')
    return (_studentEmail && _studentEmail.includes('@')) ? '✉ ' + _studentEmail : '';
  if (key === 'ip')
    return _clientIp ? 'IP ' + _clientIp : '';
  if (key === 'code')
    return _studentCode || 'CDP-?????';
  if (key === 'datetime') {
    const now = new Date();
    return now.toLocaleDateString() + ' ' + now.toLocaleTimeString();
  }
  return '';
}

// IP pública independiente (fail-safe: si falla, la marca IP queda vacía)
function fetchClientIp() {
  if (_clientIp) return;
  fetch('https://api.ipify.org?format=json', { cache: 'no-store' })
    .then(r => r.ok ? r.json() : null)
    .then(d => { if (d && d.ip) { _clientIp = d.ip;
      const el = document.getElementById('wm-ip'); if (el) el.textContent = wmValue('ip'); } })
    .catch(() => {});
}
```

3.4 · ★ Suscripción SSE — el corazón del tiempo real ★

```
let _apiBase = '', _authToken = null, _wmCourseId = null, _wmSSE = null;

function subscribeWmConfig() {
  if (_wmSSE || !_wmCourseId || !_authToken) return;
  if (typeof EventSource === 'undefined') return;

  const url = _apiBase + '/api/watermark/stream/' + encodeURIComponent(_wmCourseId)
    + '?token=' + encodeURIComponent(_authToken)
    + '&os=' + encodeURIComponent(_wmOsKey || 'web');

  const es = new EventSource(url);
  _wmSSE = es;

  // Cada vez que el admin cambia el tamaño → llega aquí y re-renderiza
  es.addEventListener('config', (e) => {
    try {
      const full = JSON.parse(e.data);
      _wmConfig = full[_wmOsKey] || full || _wmConfig; // toma la clave de SU SO
      renderWatermark(); // ← re-render sin recargar
    } catch {}
  });

  es.onerror = () => { /* EventSource reintenta solo */ };
}
```

3.5 · Inicialización (unir todo)

```
function initWatermark(opts) {
  const { apiBase, token, courseId, studentEmail, studentCode,
    watermarkConfig, os } = opts || {};
  _apiBase      = apiBase || _apiBase;
  _authToken    = token   || _authToken;
  _wmCourseId   = courseId ?? _wmCourseId;
  _studentEmail = studentEmail || _studentEmail;
  _studentCode  = studentCode || _studentCode;
  _wmOsKey      = os || 'web';
  // La config inicial ya viene resuelta por S0 desde el backend
  if (watermarkConfig) _wmConfig = watermarkConfig[_wmOsKey] || watermarkConfig;

  fetchClientIp();           // 1) IP pública async
  renderWatermark();         // 2) pintar con el tamaño actual
  subscribeWmConfig();       // 3) abrir canal tiempo real
}

// Llamada desde el reproductor al obtener la licencia del video:
initWatermark({
  apiBase:      API_BASE,
  token:        STATE.token,
  courseId:     resp.courseId,
  studentEmail: parseJwt(STATE.token).email,
  studentCode:  resp.studentCode,
  watermarkConfig: resp.watermarkConfig, // config inicial del backend
  os:           'web',                  // 'windows' | 'android' | 'ios' ...
});
```

3.6 · HTML mínimo del reproductor

```
<div id="player-container" style="position:relative">
  <video id="video" controls></video>
  <!-- Las marcas wm-email, wm-ip, wm-code, wm-datetime
        se crean por JS dentro de este contenedor -->
</div>
```

El contenedor **debe** ser `position:relative` para que las marcas (`position:absolute`) se posicionen respecto al video y no a la página.

4. Panel Admin — el slider del tamaño

El control que el usuario mueve para cambiar el tamaño en píxeles en vivo.

4.1 · Render del editor (slider 6–80 px)

```
function wmRenderMarks() {
  const cfg = _wmEnsureCourse(_wmCourse);
  const osCfg = cfg[_wmOs];
  const box = $('wm-marks');

  box.innerHTML = _wmData.marks.map(mk => {
    const m = osCfg[mk] || { on:true, size:16, color:'#ffffff', weight:600 };
    return `
    <div class="mark-card">
      <b>${_WM_MARK_LABEL[mk]}</b>
      <label><input type="checkbox" ${m.on?'checked':''}
        onchange="wmSetMark('${mk}','on',this.checked)"> Visible</label>

      <!-- ★ SLIDER DEL TAMAÑO EN PÍXELES ★ -->
      <label>Tamaño: <b id="wm-size-lbl-${mk}">${m.size}px</b>
        <input type="range" min="6" max="80" value="${m.size}"
          oninput="wmSetMark('${mk}','size',this.value)">
      </label>

      <label>Color:
        <input type="color" value="${m.color}"
          onchange="wmSetMark('${mk}','color',this.value)">
      </label>

      <label>Grosor:
        <select onchange="wmSetMark('${mk}','weight',this.value)">
          <option value="400">Normal</option>
          <option value="600">Semi-negrita</option>
          <option value="700">Negrita</option>
          <option value="900">Extra negrita</option>
        </select>
      </label>
    </div>`;
  }).join('');

  wmRenderPreview(); // vista previa en vivo
}
```

4.2 · Cambio de tamaño + auto-guardado con debounce

```
function wmSetMark(mk, field, val) {
  const m = _wmEnsureCourse(_wmCourse)[_wm0s][mk];
  if (field === 'on')    m.on = !!val;
  if (field === 'size')  { m.size = parseInt(val,10)||16;
    const l = $('wm-size-lbl-'+mk); if (l) l.textContent = m.size+'px'; }
  if (field === 'color') m.color = val;
  if (field === 'weight') m.weight = parseInt(val,10)||600;
  wmRenderPreview();
  wmScheduleAutoSave();    // ← dispara guardado + broadcast
}

// Guarda solo tras 700ms sin cambios (no satura el servidor con cada píxel)
let _wmSaveTimer = null;
function wmScheduleAutoSave() {
  clearTimeout(_wmSaveTimer);
  const course = _wmCourse;
  _wmSaveTimer = setTimeout(() => _wmPersist(course), 700);
}

async function _wmPersist(course) {
  const cfg = _wmEnsureCourse(course);
  const r = await api('POST',
    '/api/admin/watermark/config/' + encodeURIComponent(course), { config: cfg });
  _wmData.configs[course] = r.config;
  // El backend ya hizo broadcast → los reproductores ya cambiaron de tamaño
}
```

4.3 · Vista previa en vivo (mismo cálculo que el reproductor)

```
function wmRenderPreview() {
  const osCfg = _wmEnsureCourse(_wmCourse)[_wm0s];
  const box = $('wm-preview');
  const pos = ['top:10px;left:14px', 'top:10px;right:14px',
    'bottom:10px;left:14px', 'bottom:10px;right:14px'];
  box.innerHTML = _wmData.marks.map((mk,i) => {
    const m = osCfg[mk]; if (!m || !m.on) return '';
    return `<div style="position:absolute;${pos[i]};
      font-size:${m.size}px;color:${m.color};font-weight:${m.weight};
      text-shadow:0 1px 3px #000">${_WM_SAMPLE[mk]}</div>`;
  }).join('');
}
```

5. Cómo implementarlo en OTRO reproductor / otra página

Guía práctica para portar exactamente esta función a un proyecto nuevo.

1 Backend — copiar la lógica de config + SSE

- Copia las funciones de la sección 2 (`WM_MARKS` , `_wmClampInt` , `sanitizeWmConfig` , `resolveWmConfig` , `loadWmConfigs` / `saveWmConfigs`).
- Copia el `Map _wmStreams` + `_wmSubscribe/_wmUnsubscribe/broadcastWmConfig` .
- Copia las 3 rutas: `GET /stream/:courseId` , `GET /admin/.../configs` , `POST /admin/.../config/:courseId` .
- Persistencia: cualquier store con `getConfig/setConfig` (SQLite, Postgres, Redis, o incluso un JSON en disco).

2 Reproductor nuevo — pegar el módulo cliente

- Envuelve un contenedor `position:relative` alrededor del `<video>` (o del div del player: Video.js, Plyr, Shaka, HLS.js, etc.).
- Copia `_wmMarkStyle` , `WM_DEFS` , `renderWatermark` , `wmValue` , `subscribeWmConfig` , `initWatermark` (sección 3).
- Llama a `initWatermark({ ... })` justo después de iniciar la reproducción, pasando la `watermarkConfig` inicial y el `token` .
- Cambia `os` según la plataforma del nuevo player (`'web'` , `'android'` , etc.).

3 Página / Panel — reutilizar el editor

- Copia `wmRenderMarks` , `wmSetMark` , `wmScheduleAutoSave` , `_wmPersist` , `wmRenderPreview` (sección 4).
- El slider `<input type="range" min="6" max="80">` es todo lo que necesita el usuario para el tamaño.

Independencia del motor de video: la marca de agua es una capa DOM superpuesta; **no depende** de HLS.js, Shaka ni del códec. Funciona igual con cualquier `<video>` , `iframe` o `canvas`, siempre que exista un contenedor `position:relative` .

Checklist de portabilidad

#	Ítem	Detalle
1	Contenedor relative	El wrapper del video en <code>position:relative</code>
2	Config inicial	Backend envía <code>watermarkConfig</code> al cargar el video
3	Auth SSE por query	<code>?token=JWT</code> (EventSource no manda headers)
4	Clamp 6–80 en server	Nunca confiar en el tamaño del cliente
5	Heartbeat 25s	Evita que proxies cierren la conexión SSE
6	proxy_buffering off	En nginx, para que el push llegue al instante
7	Re-render idempotente	Limpiar timers/els antes de volver a pintar

Alternativa sin SSE (WebSocket o polling)

Si tu infraestructura no permite SSE, el mismo diseño funciona con:

- **WebSocket:** el reproductor se une a una sala por curso; el admin emite `config` y el server hace `io.to(curso).emit('config', cfg)`.
- **Polling:** el reproductor hace `GET /api/watermark/config/:curso` cada 15–30 s y re-renderiza si cambió (más simple, menos "instantáneo").

6. Seguridad y buenas prácticas

Riesgo	Mitigación aplicada
Tamaño inválido / abuso	<code>_wmClampInt(size, 6, 80)</code> en el servidor. El valor del cliente nunca se usa sin validar.
Color inyectado (XSS)	Regex <code>/^#[0-9a-fA-F]{6}\$/</code> ; cualquier otra cosa cae al color por defecto.
SSE sin auth	Se verifica el <code>JWT</code> del query antes de abrir el canal (<code>401</code> si falla).
Solo admin edita	Middleware <code>requireAdmin</code> en las rutas de guardado/lectura del editor.
Conexiones colgadas	Heartbeat : <code>ping</code> cada 25 s y limpieza en <code>req.on('close')</code> .
Fallo de red del cliente	<code>EventSource</code> reintenta solo; la marca conserva el último tamaño conocido.

Recuerda: la marca de agua DOM es **disuasoria/forense**, no una protección criptográfica. Un usuario avanzado puede ocultarla con DevTools. Para protección fuerte, combínala con watermarking en el propio stream (servidor) y detección de DevTools.

Resumen de endpoints

Método	Ruta	Rol	Función
GET	<code>/api/watermark/stream/:courseId</code>	alumno	Canal SSE tiempo real
GET	<code>/api/watermark/config/:courseId</code>	alumno	Config puntual (polling)
GET	<code>/api/admin/watermark/configs</code>	admin	Cargar editor
POST	<code>/api/admin/watermark/config/:courseId</code>	admin	Guardar + broadcast

Ficheros de referencia en este proyecto

- **server.js** — helpers de config, SSE y rutas (secciones 2–3 de esta guía).
- **public/js/security.js** — módulo del reproductor (`initWatermark` , `_subscribeWmConfig`).
- **index.html** — estilos `#watermark-overlay` y llamada `CDP_SECURITY.init()` .
- **admin.html** — editor con slider de tamaño (`wmRenderMarks` , `wmSetMark`).